

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracemarks) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Общее задание

Задание 1. Базовый перехват запуска процессов

Задание 2. Динамический анализ аномального поведения (Индивидуальный вариант)

Задание 3. Интеграция с пространством пользователя и аудит

Задание 4. Проектирование профилей защиты (Без написания кода)

Вариант	Целевой системный вызов (или tracepoint)	Условие аномального поведения (Фильтр ядра)
3	sys_enter_fchmodat	Попытка установить права 0777 на любой файл

Ход работы

Задание 1.

1. Разработайте eBPF-зонд (C + Python), использующий механизм перехвата kprobe (или tracepoint) для отслеживания системного вызова execve (запуск новых процессов).

```
def print_event(cpu, data, size):
    # Конвертируем сырые байты из ядра в структуру Python
    event = bpf["events"].event(data)

    # Задания 1 и 3. ВАШ КОД ЗДЕСЬ:
    # 1. Выведите в консоль event.pid, event.uid и event.comm.decode('utf-8', 'replace')
    # 2. (для Задания 3): Добавьте логику записи этих данных в файл security_audit.log

    # Пример вывода (замените на свой):
    print(f"PID: {event.pid:<7} UID: {event.uid:<6} COMM: {event.comm.decode('utf-8', 'replace')}")
```

2. Напишите логику на стороне ядра (C), которая извлекает идентификатор процесса (PID), идентификатор пользователя (UID) и имя исполняемого файла (comm).

```
int syscall_execve(struct pt_regs *ctx) {
    struct data_t data = {};

    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    events.perf_submit(ctx, &data, sizeof(data));

    return 0;
}
```

3. Запустите скрипт с правами root и продемонстрируйте перехват запуска стандартных утилит (например, ls или cat) в соседнем окне терминала или новой вкладке WSL.

```
Ubuntu@WIN-ARCOSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ cd "/mnt/d/уник/4.1 курс/СМЗКС/lab3"
sudo python3 task1.py
[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)
PID: 11352 UID: 0 COMM: containerd-shim FILE: /usr/bin/runc
PID: 11362 UID: 0 COMM: runc FILE: /proc/self/fd/6
PID: 11364 UID: 101 COMM: runc:[2:INIT] FILE: /bin/sh
PID: 11370 UID: 101 COMM: sh FILE: /usr/local/bin/healthcheck
PID: 11372 UID: 101 COMM: healthcheck FILE: /usr/bin/env
PID: 11373 UID: 101 COMM: healthcheck FILE: /usr/bin/grep
PID: 11374 UID: 101 COMM: healthcheck FILE: /usr/bin/curl
PID: 11376 UID: 0 COMM: cron FILE:
PID: 11377 UID: 0 COMM: cron FILE: /bin/sh
PID: 11378 UID: 0 COMM: sh FILE: /usr/bin/run-parts
PID: 11385 UID: 1000 COMM: bash FILE: /usr/bin/sed
PID: 11390 UID: 1000 COMM: bash FILE: /usr/bin/sed
PID: 11391 UID: 1000 COMM: bash FILE: /usr/bin/ls
PID: 11396 UID: 1000 COMM: bash FILE: /usr/bin/sed
PID: 11397 UID: 1000 COMM: bash FILE: /usr/bin/cat
PID: 11401 UID: 1000 COMM: bash FILE: /usr/bin/sed
^C
```

Задание 2.

1. Определите целевой системный вызов и условие аномального поведения согласно вашему индивидуальному варианту.

- целевой системный вызов: `sys_enter_fchmodat`;
- условие аномалии: попытка установить права 0777 на любой файл.

2. Напишите eBPF-зонд на C, извлекающий аргументы выбранного системного вызова (например, путь к целевому файлу, целевой PID или сетевой порт).

```
#!/usr/bin/python3
from bcc import BPF

bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;
    u32 mode;
    char comm[TASK_COMM_LEN];
    char fname[128];
};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_fchmodat) {
    // ===== ФИЛЬТРАЦИЯ В ЯДРЕ (начало) =====
    if ((args->mode & 0777) != 0777)
        return 0;
    // ===== ФИЛЬТРАЦИЯ В ЯДРЕ (конец) =====

    struct data_t data = {};
    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    data.mode = args->mode & 0777;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    bpf_probe_read_user_str(&data.fname, sizeof(data.fname), args->filename);

    events.perf_submit(args, &data, sizeof(data));
    return 0;
}
"""

b = BPF(text=bpf_text)

print("[*] Монитор запущен. Слежу за fchmodat с правами 0777... (Ctrl+C для выхода)")

def print_event(cpu, data, size):
    e = b["events"].event(data)
    print(f"[!] АЛЕПТ: попытка выставить права 0777! "
          f"PID={e.pid} UID={e.uid} mode={e.mode:04o} "
          f"COMM={e.comm.decode('utf-8', 'replace')} "
          f"FILE={e.fname.decode('utf-8', 'replace')}")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

3. Реализуйте логику фильтрации строго на стороне ядра: программа должна передавать событие в пространство пользователя (через кольцевой буфер BPF_PERF_OUTPUT) только при

совпадении с условием аномалии. Передача всех подряд событий в Python-скрипт является грубой ошибкой производительности.

```
TRACEPOINT_PROBE(syscalls, sys_enter_fchmodat) {
    if ((args->mode & 0777) != 0777)
        return 0;

    struct data_t data = {};
    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    data.mode = args->mode & 0777;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    bpf_probe_read_user_str(&data.fname, sizeof(data.fname), args->filename);

    events.perf_submit(args, &data, sizeof(data));
    return 0;
}
```

4. Сгенерируйте целевую аномалию (например, попытайтесь прочитать защищенный файл) и зафиксируйте успешный перехват алерта вашим скриптом.

```
Ubuntu@WIN-ARCSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ touch /tmp/test.txt
chmod 644 /tmp/test.txt
chmod 777 /tmp/test.txt

Ubuntu@WIN-ARCSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ cd "/mnt/d/уник/4.1 курс/СМЗКС/lab3"
sudo python3 task2.py
[*] Монитор запущен. Слежу за fchmodat с правами 0777... (Ctrl+C для выхода)
[!] АЛЕРТ: попытка выставить права 0777! PID=12366 UID=1000 mode=0777 COMM=chmod FILE=/tmp/test.txt
```

Задание 3.

1. Расширьте Python-скрипт из задания 2: при получении события об аномалии из ядра скрипт должен автоматически записывать предупреждение в локальный лог-файл security_audit.log.

```
#!/usr/bin/python3
from bcc import BPF
from datetime import datetime

LOG_FILE = "security_audit.log"

bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;
    u32 mode;
    char comm[TASK_COMM_LEN];
    char fname[128];
};

BPF_PERF_OUTPUT(events);

static inline int check_and_send(void *ctx, u32 mode, const char __user *filename) {
    // Фильтрация в ядре: только права 0777
    if ((mode & 0777) != 0777)
        return 0;

    struct data_t data = {};
    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    data.mode = mode & 0777;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    bpf_probe_read_user_str(&data.fname, sizeof(data.fname), filename);

    events.perf_submit(ctx, &data, sizeof(data));
    return 0;
}
```

```

TRACEPOINT_PROBE(syscalls, sys_enter_fchmodat) {
    return check_and_send(args, args->mode, args->filename);
}

TRACEPOINT_PROBE(syscalls, sys_enter_fchmodat2) {
    return check_and_send(args, args->mode, args->filename);
}

TRACEPOINT_PROBE(syscalls, sys_enter_chmod) {
    return check_and_send(args, args->mode, args->filename);
}
"""

b = BPF(text=bpf_text)

print("[*] Монитор с аудитом запущен. (Ctrl+C для выхода)")

def print_event(cpu, data, size):
    e = b["events"].event(data)
    ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    line = (f"[{ts}] ALERT PID={e.pid} UID={e.uid} "
           f"COMM={e.comm.decode('utf-8', 'replace')} "
           f"EVENT=fchmodat MODE={e.mode:04o} "
           f"FILE={e.fname.decode('utf-8', 'replace')} (chmod 0777 attempt)")
    print(line)
    with open(LOG_FILE, "a") as f:
        f.write(line + "\n")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()

```

2. Формат записи должен содержать: временную метку (Timestamp), PID нарушителя, UID нарушителя и детали события.

```

Ubuntu@WIN-ARCOSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ touch /tmp/test.txt
chmod 777 /tmp/test.txt
touch /tmp/b.txt
chmod 777 /tmp/b.txt
chmod 644 /tmp/b.txt

Ubuntu@WIN-ARCOSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ cd "/mnt/d/уник/4.1 курс/СМЗКС/lab3"
sudo python3 task3.py
[*] Монитор с аудитом запущен. (Ctrl+C для выхода)
[2026-10-07 13:34:12] ALERT PID=12840 UID=1000 COMM=chmod EVENT=fchmodat MODE=0777 FILE=/tmp/b.tx
t (chmod 0777 attempt)

Ubuntu@WIN-ARCOSMBQB99:/mnt/d/уник/4.1 курс/СМЗКС/lab3$ cat security_audit.log
[2026-10-07 13:34:12] ALERT PID=12840 UID=1000 COMM=chmod EVENT=fchmodat MODE=0777 FILE=/tmp/b.tx
t (chmod 0777 attempt)

```

Задание 4.

1. Сравните механизмы изоляции Seccomp-BPF и eBPF (kprobes) применительно к задаче блокировки системных вызовов. Обоснуйте, почему стандартный eBPF-зонд преимущественно используется для мониторинга и аудита, а не для предотвращения (блокировки) действий вредоносного ПО.

1) Seccomp-BPF это механизм ограничения доступа. Фильтр привязывается к процессу и срабатывает в штатной точке ядра до выполнения системного вызова. Его вердикт (SECCOMP_RET_ALLOW, SECCOMP_RET_ERRNO, SECCOMP_RET_KILL) является частью логики ядра, поэтому вызов блокируется корректно. Фильтры наследуются дочерними процессами и не могут быть сняты.

2) eBPF kprobe это механизм трассировки. Зонд прикрепляется к функции ядра и наблюдает за её вызовами. Возвращаемое значение программы не влияет на ход выполнения ядра, штатного механизма отмены вызова у него нет.

3) Почему eBPF-зонд используют для мониторинга и аудита, а не для блокировки:

а. kprobe срабатывает в произвольной точке ядра, часто внутри критических секций (удержаны спинлоки, отключены прерывания, структуры обновлены наполовину), поэтому принудительный выход оставил бы ядро в несогласованном состоянии (взаимная блокировка, утечка ресурсов, паника);

б. код ядра рассчитан на определённые пути завершения и обработки ошибок, неожиданный выход нарушает эти предположения;

с. верификатор eBPF намеренно не позволяет программам влиять на ход выполнения ядра;

д. зонд работает параллельно с вызовом, поэтому реакция из пространства пользователя (Python) приходит уже после того, как действие выполнено.

4) Вывод: kprobe и tracerpoint используют для мониторинга и аудита, а блокировку делают через Sessomp или BPF LSM. В моей работе зонд на `sys_enter_fchmodat` фиксирует попытку выставить права 0777, но не запрещает её.

2. Опишите концепцию атаки состояния гонки типа TOCTOU (Time-of-Check to Time-of-Use) применительно к вашему eBPF-зонду из задания 2 и предложите архитектурный метод противодействия.

1) TOCTOU (Time-of-Check to Time-of-Use) это состояние гонки: данные проверили в один момент, а использовали в другой, и между ними они изменились.

2) В моём зонде аргумент `mode` передаётся значением в регистре, поэтому его нельзя подменить. Аргумент `filename` передаётся указателем на память процесса, и тут возможна атака:

- зонд читает строку пути при входе в вызов и видит безобидный путь, например `/tmp/safe.txt`;
- другой поток процесса перезаписывает буфер пути (например, на `/etc/shadow`) до того, как ядро копирует его для реальной работы;
- в лог попадёт `/tmp/safe.txt`, а права 0777 будут выставлены на другой файл;
- аналогичный вариант: подмена символической ссылки после проверки.

3) Архитектурные методы противодействия:

а. копировать аргумент-указатель один раз через `bpf_probe_read_user_str()` и работать только с копией;

б. контролировать не запрос, а результат: использовать BPF LSM (хук `security_path_chmod`), где ядро уже скопировало и разрешило путь и работает со своими данными, а подменить их нельзя;

с. связать `sys_enter` и `sys_exit` через BPF-карту (`BPF_HASH`) по `pid_tgid` и проверять код возврата;

д. логировать идентификаторы ядра (`inode`, устройство), а не строку пути из пользовательской памяти.

Вывод: освоила методы разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracerpoints) и выявила аномальное поведение процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).